PROJECT REPORT
BIO-CHEMICAL
INGREDIENT SAFETY RATER

1. Introduction & Problem Identification

1.1 The Problem

In the modern personal care industry, product labels are often filled with complex chemical nomenclature that is indecipherable to the average consumer. Many of these ingredients, while legally permissible in small quantities, are linked to long-term biological risks such as endocrine disruption, chronic skin irritation, and environmental toxicity (e.g., coral reef bleaching).

1.2 Why It Matters

As a Bioengineering student, I recognized that there is a significant "information gap" between biochemical research and consumer awareness. Identifying these risks manually is time-consuming and prone to error. A computational approach allows for instantaneous, objective analysis of these risks, empowering users to make informed health decisions.

2. Project Scope & Target AudienceScope:

To develop a Python-based analyzer that utilizes external data (CSV) to identify high-risk ingredients and provide a weighted safety rating (1–10).

Target Users: Health-conscious consumers, dermatologists, and bioengineering students interested in toxicology.

3. Methodology & Approach

3.1 Computational Thinking

I applied the Top-Down Design strategy learned in Unit 1. I broke the problem into four distinct sub-tasks:

a) Data Persistence:
   - Storing toxicological data in a structured format (`toxins.csv`).
b) Input Parsing:
   - Converting a raw string of ingredients into a searchable Python List.
c) Cross-Referencing:
   - Using Dictionary lookups to match user data with known risks.
d) Quantitative Analysis:
   - Implementing a scoring algorithm to provide a final safety grade.

3.2 Key Algorithms

The core logic relies on a "Fuzzy Search" within a loop. Because chemical names often vary (e.g., "Methylparaben" vs "Paraben"), the code checks if the toxin name is *contained* within the user's input string, ensuring higher accuracy.

4. Technical Implementation (Syllabus Integration)

This project serves as a practical application of several course units:

1. Control Flow:
   - Used `for` loops to iterate through lists and `if-elif-else` structures to determine the final safety status.
2. Lists:
   - Used to store and clean the user's input.
3. Dictionaries:
   - Used to store "Compound Data" (Ingredient name as key; Risk and Score as values).
4. File Handling:
   - Implemented CSV reading to demonstrate how programs interact with external data environments.

5. Key Decisions & Challenges

5.1 Decision: Weighted Scoring

I decided against a simple "Yes/No" toxic check. Instead, I assigned each toxin a score from 1–10 based on its biological severity (e.g., a known carcinogen like Formaldehyde is a 10, while a mild irritant like SLS is a 3). This provides a more nuanced and "engineered" result.

5.2 Challenge: Handling Delimiters

Users often copy-paste lists with irregular spacing or varying capitalization. Solution: I utilized Python's .strip() and .lower() methods during the list comprehension phase to "sanitize" the data before analysis, ensuring the program doesn't crash due to formatting errors.

6. What I Learned

Through this BYOP, I learned that:

a) Abstraction is Key:
   - Programming isn't just about writing code; it's about taking a complex biological problem and identifying which parts can be solved with logic.

b) Data Integrity:
   - A program is only as good as its data. Moving the toxin list to an external CSV file showed me how to make software scalable and easier to update.

c) User-Centric Design:
   - Providing a final "1–10" score is much more useful to a human user than just listing chemical names.

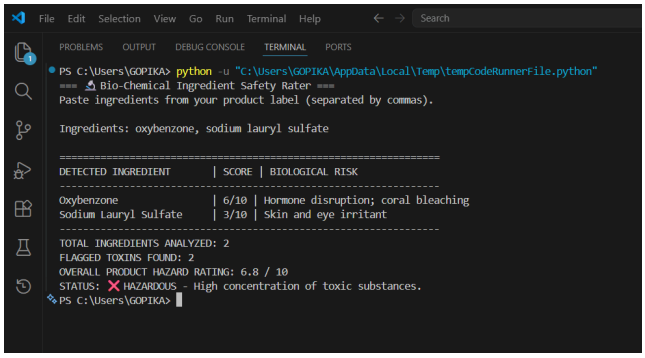
7.Code

```python
import csv

# STEP 1: Create the expanded Toxin Database file
# This includes common cosmetic toxins, allergens, and irritants
csv_content = """ingredient,risk,score
paraben,Endocrine disruptor; mimics estrogen,7
phthalate,Reproductive system toxicity,8
formaldehyde,Known human carcinogen,10
triclosan,Thyroid hormone disruption,6
sodium lauryl sulfate,Skin and eye irritant,3
toluene,Immune system toxin,9
oxybenzone,Hormone disruption; coral bleaching,6
fragrance,Potential hidden allergens/phthalates,5
lead,Heavy metal; neurotoxicity,10
hydroquinone,Skin bleacher; organ system toxicity,9
polyethylene glycol,"Can be contaminated with 1,4-dioxane",4
bha,Potential endocrine disruptor and carcinogen,8
bht,Skin irritant and potential hormone disruptor,5
coal tar,Known human carcinogen,10
synthetic colors,Derived from petroleum; skin irritant,4
"""

with open('toxins.csv', 'w') as f:
    f.write(csv_content)

def load_toxin_database(filename):
    """Syllabus Unit 5: Load compound data into a Dictionary"""
    database = {}
    try:
        with open(filename, mode='r', encoding='utf-8') as file:
            reader = csv.DictReader(file)
            for row in reader:
                database[row['ingredient'].lower()] = {
                    'risk': row['risk'],
                    'score': int(row['score'])
                }
        return database
```

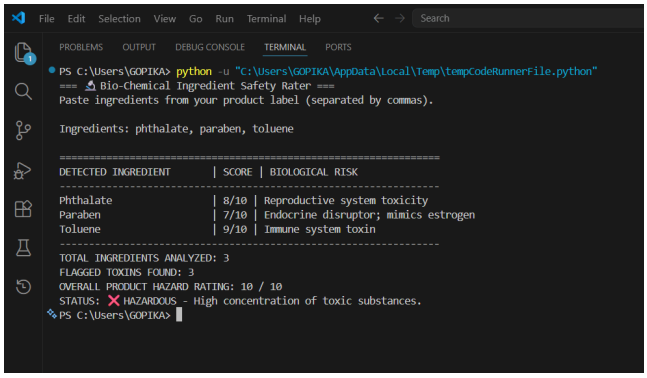
```python
    except FileNotFoundError:
        print(f"Error: {filename} not found.")
        return None

def analyze_and_rate():
    toxin_db = load_toxin_database('toxins.csv')
    if not toxin_db: return

    print("=== 🔬 Bio-Chemical Ingredient Safety Rater ===")
    print("Paste ingredients from your product label (separated by commas).")
    raw_input = input("\nIngredients: ")

    # Syllabus Unit 5: List operations
    user_list = [item.strip().lower() for item in raw_input.split(',')]

    total_hazard_score = 0
    flagged_items = []

    print("\n" + "="*65)
    print(f"{'DETECTED INGREDIENT':<25} | {'SCORE':<5} | {'BIOLOGICAL RISK'}")
    print("-" * 65)

    # Syllabus Unit 3: Iteration (for loops) and Conditionals (if)
    for item in user_list:
        for toxin, data in toxin_db.items():
            if toxin in item:
                print(f"{item.title():<25} | {data['score']}/10 | {data['risk']}")
                total_hazard_score += data['score']
                flagged_items.append(item)
                break

    # --- FINAL RATING CALCULATION ---
    if len(user_list) > 0:
        # We calculate the weighted hazard based on the whole formula
        # Final Rating = (Sum of Toxin Scores) / (Total Ingredients)
```

```python
def analyze_and_rate():

    # --- FINAL RATING CALCULATION ---
    if len(user_list) > 0:
        # We calculate the weighted hazard based on the whole formula
        # Final Rating = (Sum of Toxin Scores) / (Total Ingredients)
        product_rating = (total_hazard_score / len(user_list)) * 1.5

        # Cap the rating between 0 and 10
        final_score = round(min(max(product_rating, 0.5), 10), 1)

        print("-" * 65)
        print(f"TOTAL INGREDIENTS ANALYZED: {len(user_list)}")
        print(f"FLAGGED TOXINS FOUND: {len(flagged_items)}")
        print(f"OVERALL PRODUCT HAZARD RATING: {final_score} / 10")

        # Syllabus Unit 3: Conditional flow (if-elif-else)
        if final_score <= 2.5:
            print("STATUS: ✅ CLEAN - Minimal to no toxic ingredients found.")
        elif 2.5 < final_score <= 5.5:
            print("STATUS: ⚠️ CAUTION - Moderate levels of irritants or disruptors.")
        else:
            print("STATUS: ❌ HAZARDOUS - High concentration of toxic substances.")
    else:
        print("Error: No ingredients detected for analysis.")

# Execute the program
if __name__ == "__main__":
    analyze_and_rate()
```

Output



```
PS C:\Users\GOPIKA> python -u "C:\Users\GOPIKA\AppData\Local\Temp\tempCodeRunnerFile.python"
=== Bio-Chemical Ingredient Safety Rater ===
Paste ingredients from your product label (separated by commas).

Ingredients: oxybenzone, sodium lauryl sulfate

==============================================================
DETECTED INGREDIENT        | SCORE | BIOLOGICAL RISK
--------------------------------------------------------------
Oxybenzone                 | 6/10 | Hormone disruption; coral bleaching
Sodium Lauryl Sulfate      | 3/10 | Skin and eye irritant
--------------------------------------------------------------
TOTAL INGREDIENTS ANALYZED: 2
FLAGGED TOXINS FOUND: 2
OVERALL PRODUCT HAZARD RATING: 6.8 / 10
STATUS: ❌ HAZARDOUS - High concentration of toxic substances.
PS C:\Users\GOPIKA>
```



```
PS C:\Users\GOPIKA> python -u "C:\Users\GOPIKA\AppData\Local\Temp\tempCodeRunnerFile.python"
=== Bio-Chemical Ingredient Safety Rater ===
Paste ingredients from your product label (separated by commas).

Ingredients: phthalate, paraben, toluene

==============================================================
DETECTED INGREDIENT        | SCORE | BIOLOGICAL RISK
--------------------------------------------------------------
Phthalate                  | 8/10 | Reproductive system toxicity
Paraben                    | 7/10 | Endocrine disruptor; mimics estrogen
Toluene                    | 9/10 | Immune system toxin
--------------------------------------------------------------
TOTAL INGREDIENTS ANALYZED: 3
FLAGGED TOXINS FOUND: 3
OVERALL PRODUCT HAZARD RATING: 10 / 10
STATUS: ❌ HAZARDOUS - High concentration of toxic substances.
PS C:\Users\GOPIKA>
```

9. Conclusion

The Bio-Chemical Ingredient Safety Rater successfully demonstrates that introductory programming concepts, like dictionaries and loops, can solve contemporary engineering challenges. This project provides a foundation that could eventually be expanded into a mobile app or a web-based database for global consumer safety.